

3.6.5 Temel örnekler

Django, geniş bir uygulama yelpazesine uygun temel görünüm sınıfları sağlar. Tüm görünüm, görünümün URL'lere bağlanmasını, HTTP metodu yönlendirmesini ve diğer yaygın özellikleri işleyen View sınıfından miras alır. `RedirectView` bir HTTP yönlendirmesi sağlar ve `TemplateView`, bir şablonu da işlemesini sağlamak için temel sınıfı genişletir.

3.6.6 URLconf'unuzda kullanım

Genel görünüm, görünüm kullanmanın en doğrudan yolu, onları doğrudan URLconf'unuzda oluşturmaktır. Sınıf tabanlı bir görünümde yalnızca birkaç niteliği değiştiriyorsanız, bunları `as_view()` metod çağrısının kendisine iletebilirsiniz:

```
from django.urls import path
from django.views.generic import TemplateView

urlpatterns = [
    path("about/", TemplateView.as_view(template_name="about.html")),
]
```

`as_view()` metoduna iletilen herhangi bir argüman, sınıfta ayarlanan niteliklerin üzerine yazacaktır. Bu örnekte, `TemplateView` üzerinde `template_name` niteliğini ayarlıyoruz. Benzer bir üzerine yazma deseni, `RedirectView` üzerindeki `url` niteliği için de kullanılabilir.

3.6.7 Genel görünüm, görünüm alt sınıflara ayırma

Genel görünüm, görünüm kullanmanın ikinci ve daha güçlü yolu, mevcut bir görünüm, görünüm miras almak ve yeni değerler veya metodlar sağlamak için alt sınıfınızdaki nitelikleri (`template_name` gibi) veya metodları (`get_context_data` gibi) geçersiz kılmaktır. Örneğin, sadece bir şablon olan `about.html` dosyasını görüntüleyen bir görünüm düşünün. Django'nun bunu yapacak genel bir görünüm vardır - `TemplateView` - böylece onu alt sınıfa ayırabilir ve şablon adını geçersiz kılabiliriz:

```
# some_app/views.py
from django.views.generic import TemplateView

class AboutView(TemplateView):
    template_name = "about.html"
```

Ardından bu yeni görünümü URLconf'umuza eklememiz gerekir. `TemplateView` bir fonksiyondan ziyade bir sınıftır, bu yüzden URL'yi doğrudan sınıf metodu olan `as_view()` 'e yönlendiririz; bu, sınıf tabanlı görünüm, görünüm fonksiyon benzeri bir giriş noktası sağlar:

```
# urls.py
from django.urls import path
from some_app.views import AboutView

urlpatterns = [
    path("about/", AboutView.as_view()),
]
```

Yerleşik genel görünüm, görünüm nasıl kullanılacağı hakkında daha fazla bilgi için, genel sınıf tabanlı görünüm, görünüm hakkındaki bir sonraki konuya bakın.

Diğer HTTP metodlarını destekleme

Birinin kitap kütüphanemize HTTP kullanarak bir API gibi erişmek istediğini varsayalım. API istemcisi arada bir bağlanacak ve son ziyaretten bu yana yayınlanan kitapların verilerini indirecektir. Ancak o zamandan beri yeni kitap eklenmemişse, bir yanıt oluşturmak ve bunu istemciye göndermek CPU süresi ve bant genişliği israfıdır. En son kitabın ne zaman yayınlandığını API'ye sormak daha tercih edilebilir olabilir.

URLconf'ta URL'yi kitap listesi görünümüne eşliyoruz:

```
from django.urls import path
from books.views import BookListView

urlpatterns = [
    path("books/", BookListView.as_view()),
]
```

Ve görünüm:

```
from django.http import HttpResponse
from django.views.generic import ListView
from books.models import Book

class BookListView(ListView):
    model = Book

    def head(self, *args, **kwargs):
        last_book = self.get_queryset().latest("publication_date")
        response = HttpResponse(
            # RFC 1123 date format.
            headers={
                "Last-Modified": last_book.publication_date.strftime(
                    "%a, %d %b %Y %H:%M:%S GMT"
                )
            },
        )
        return response
```

Görünüme bir GET isteği ile erişilirse, yanıtta bir nesne listesi döndürülür. Ancak istemci bir HEAD isteği gönderirse, yanıtın gövdesi boş olur ve Last-Modified başlığı en son kitabın ne zaman yayınlandığını belirtir.

3.6.8 Asenkron sınıf tabanlı görünümler

Halihazırda gösterilen senkron (def) metod işleyicilerinin yanı sıra, View alt sınıfları, await kullanarak asenkron koddan yararlanmak için asenkron (async def) metod işleyicileri tanımlayabilir:

```
import asyncio
from django.http import HttpResponse
from django.views import View

class AsyncView(View):
    async def get(self, request, *args, **kwargs):
        # Perform io-blocking view logic using await, sleep for example.
        await asyncio.sleep(1)
        return HttpResponse("Hello async world!")
```

Tek bir görünüm sınıfı içinde, tüm kullanıcı tanımlı metod işleyicileri ya senkron ya da asenkron olmalıdır. Django, asenkron görünümeleri otomatik olarak algılayacak ve bunları asenkron bir bağlamda çalıştıracaktır.